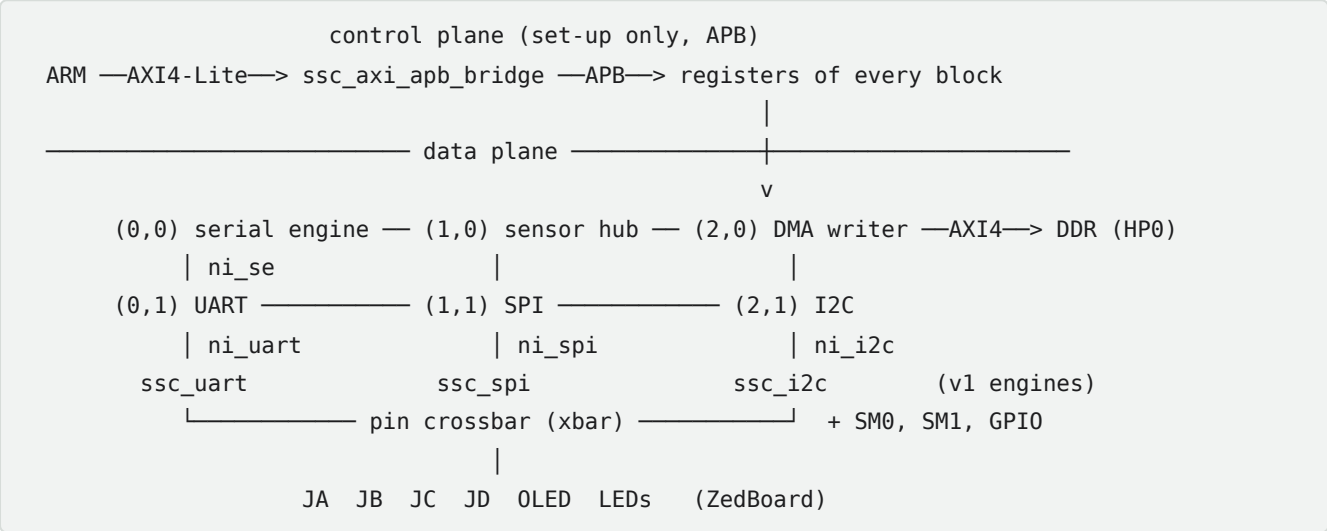
Code walkthrough: every module of v2

This document explains how the v2 code is organised and what every module does, then follows one temperature reading and one flash-ID request through the real modules. Each block also has its own detailed explanation in `docs/blocks/` (ports, registers, timing diagrams, how its testbench proves it), and `docs/SPEC.md` is the contract the blocks were built against.

1. The big picture

v2 keeps the v1 idea (one controller for UART, SPI and I2C on the ZedBoard) but changes how data moves:



- The **control plane** is the v1 path: the CPU writes registers through an AXI-to-APB bridge. In v2 the CPU uses it once, to set things up.
- The **data plane** is a 2 x 3 network-on-chip. Six nodes exchange packets: three network interfaces in front of the v1 engines, the serial engine, the sensor hub and the DMA writer. Protocol conversion is just a packet from one node to another.
- The **pin crossbar** connects any engine (or a serial-engine state machine, or plain GPIO) to any connector, and can switch while running.

2. Folder map

```
smart_serial_controller_v2/
  rtl/v1/      the v1 engines and helpers, reused (only the ID changed)
  rtl/noc/     network: packet helpers, router, mesh, UART/SPI/I2C network interfaces
  rtl/se/      serial engine: state machine, engine, its network interface
  rtl/xbar/    pin crossbar
  rtl/hub/     sensor hub and DMA writer
  rtl/top/     the core that wires everything, AXI wrapper, time base,
               FPGA-only boot sequencer and record RAM
  boards/zedboard/  pads, the two top levels, pin constraints (XDC)
  programs/    serial-engine programs (.se) and their assembled .hex/.lst
  sw/          bare-metal C for the ARM (driver, demo console, demos)
  tools/       assembler, program builder, PC frame tool, font and area scripts
  sim/         testbenches: sim/unit (one per block), the two system tests, models
  vivado/      Tcl scripts: build Way 1, build Way 2 (+ XSA), run a simulation
  windows/     double-click .bat files for the Tcl scripts
  noc_study/   the bus / crossbar / mesh comparison (slide 24, measurement 6)
  docs/        this file, SPEC, register map, verification, measurements, blocks/
```

3. Every module

3.1 Reused from v1 (`rtl/v1/`)

module	what it does
`ssc_uart`	UART with 16-byte FIFOs, 5-8 data bits, parity, 1/2 stop bits, RTS/CTS, fractional baud generator
`ssc_spi`	SPI master/slave, modes 0-3, 4-32-bit words, 4 chip selects, manual/automatic CS, 16-word FIFOs
`ssc_i2c`	I2C master, 7/10-bit addresses, repeated START, clock stretching, arbitration-loss detection, 16-byte FIFOs
`ssc_fifo`, `ssc_baud_gen`, `ssc_sync_filter`	first-word-fall-through FIFO, fractional baud tick, 2-flop synchroniser + glitch filter
`ssc_regbank`	the v1 registers (0x000-0x3FF); `ID_VALUE` is now a parameter (v2 passes 0x53534302)
`ssc_irq`	event flags with write-1-to-clear and an enable mask; used twice in v2 (v1 flags and INT2)
`ssc_axi_apb_bridge`	AXI4-Lite slave to APB master (one transfer at a time)

These are explained in the v1 project; v2 does not change their behaviour.

3.2 Network (`rtl/noc/`, `docs/blocks/noc.md`)

module	what it does
`noc_pkt_tx`	turns "header + bytes" into flits: HEAD (type, dest, source, prio, length, tag, arg), BODY/TAIL with 4 bytes each, or one SINGLE flit. Keeps the credit counter, so it never sends into a full buffer
`noc_pkt_rx`	the reverse: a 4-flit buffer, header fields for the node, then the payload one byte at a time; returns one credit per freed slot
`noc_router`	5-port router (local, N, E, S, W): input FIFOs, XY routing, wormhole lock, priority then round-robin arbitration, credit counters, registered outputs, 2 clocks per hop
`noc_mesh`	W x H routers wired to their neighbours; boundary ports tied off
`ni_uart`	node 3: PC bytes from the UART RX FIFO become packets (RAW, FRAME `[wlen][w][rlen]`, ADDRESSED); every payload byte that arrives goes to the UART TX FIFO
`ni_spi`	node 4: runs an XFER_REQ on the SPI engine (CS low, write wlen, read rlen, CS high) and replies; DATA = one write frame
`ni_i2c`	node 5: runs an XFER_REQ on the I2C engine (write, repeated START, read, NACK/arbitration/time-out handling) and replies with a status byte

3.3 Serial engine (`rtl/se/`, `docs/blocks/serial_engine.md`, `se_programs.md`, `ni_se.md`)

module	what it does
`se_sm`	one state machine: PC, X, Y, OSR/ISR with shift counters, CRC-16, delay and side-set, fractional clock divider, EXEC/RESTART, 4 pins (out, oe, synchronised in)
`se_engine`	two `se_sm`, the shared 32 x 16-bit program memory, 4-word TX/RX FIFOs per SM, the registers at 0x800, CPU and network access to the FIFOs
`ni_se`	node 0: network bytes -> TX words of SM arg[0]; RX words -> DATA packets (1 or 4 bytes per word, up to 16 per packet)

The instruction set has 8 opcodes (JMP, WAIT, IN, OUT, PUSH/PULL, SET, OD, CRC). OD (open drain) and CRC are the additions for serial buses.

3.4 Pin crossbar (`rtl/xbar/xbar.v`, `docs/blocks/xbar.md`)

One select register per port. A switch waits until the old source is idle (so a frame is never cut), floats the pins for one clock, hands over, and keeps the new source's inputs idle for 8 clocks. `SW_CYCLES` and `SW_EDGE` measure every switch (measurement 3).

3.5 Sensor hub and DMA writer (`rtl/hub/`, `docs/blocks/hub.md`, `dma_writer.md`)

module	what it does
`hub`	8 tasks with period/phase timers (or "every N records"); a task sends an XFER_REQ or DATA packet, waits for the reply with the same tag, keeps the last value and makes a 16-byte time-stamped RECORD for the DMA writer
`dma_writer`	takes RECORD packets and writes each one as a 4-beat AXI4 burst into a ring buffer in DDR; counts records and raises a batch event after 32 (or after a time-out)

3.6 Integration (`rtl/top/`)

`ssc2_core.v`: the whole controller. It instantiates every block and does five jobs:

1. **Address decode.** The 12-bit APB address selects one region by its top 4 bits (`page`): 0-3 v1 registers, 4 crossbar, 5 NI configuration, 6 hub, 7 DMA, 8-9 serial engine, A misc. Each block gets one-clock `reg_we` / `reg_re` strobes and the offset inside its region; `prdata` is a mux on `page`.
2. **Engine sharing.** The CPU (through `ssc_regbank`) and the engine's network interface use the same FIFOs. The push/pop strobes are OR-ed and the data muxed; it cannot collide because every NI keeps off the engine while `stall = psel` is 1. While the SPI or I2C NI runs a transaction, its `active` output overrides the settings it needs (8-bit words, manual CS, master mode / 7-bit address).
3. **Pin roles.** Each engine's pins are packed into the crossbar's 4-pin source nibbles by the role table (UART: CTS#, TXD, RXD, RTS#; SPI: CS0#, MOSI, MISO, SCK, reversed in slave mode; I2C: SCL and SDA as open-drain `oe`). The core also tells the crossbar when each engine is idle (`src_idle`) and what its inputs should see while disconnected (`src_in_idle`, e.g. RXD high).
4. **Core registers.** NI configuration (`NI_CFG`, time-outs), packet counters for all six nodes (from the head flits on each mesh link), VERSION, TIME_US, the second interrupt register INT2, the CPU-load counters APB_COUNT / IRQ_COUNT, OLED power, BOARD_IN and SE_LOOP.
5. **Interrupt.** `irq = v1 flags & enable | INT2 flags & enable`, registered.

`ssc2_axi_top.v` wraps the core for the Zynq block design: an AXI4-Lite slave (`S_AXI_*`, through `ssc_axi_apb_bridge`) for the registers and an AXI4 master (`M_AXI_*`, write channels from the DMA writer, read channel tied off) for S_AXI_HP0. The upper-case names and `X_INTERFACE` attributes let Vivado recognise both interfaces when the module is added to the block design.

`ssc2_timebase.v` makes `time_us` (32-bit microsecond counter), a 1-clock `us_tick` and `ms_tick`. Time stamps, time-outs and the hub's periods all use it. `US_PER_MS` lets the testbenches shorten a millisecond.

`ssc2_boot.v` is the "set-up CPU" of the FPGA-only build. It is an APB master that steps through a small ROM: `WR addr, data` (write a register), `CP src -> dst` (read, shift/mask/add, write: used to copy the switches into the crossbar and NI registers), `WAIT ms` and `JMP`. The list enables the engines and NIs, loads the slide-13 UART program into SM0, sets up two hub tasks (TMP2 every 100 ms -> record; last value -> SM0) and the DMA writer, then loops every 10 ms: SW2..0 -> JD source, SW7 -> UART NI mode, record count -> LD0..3.

`ssc2_axi_bram.v` is a 4 KB AXI4 write-only block RAM: in Way 1 the records go there instead of DDR.

3.7 Board (`boards/zedboard/`)

file	what it does
`zed2_pads.v`	turns the crossbar's out/oe/in buses into tri-state pins (`pin = oe ? out : Z`); LEDs are outputs; switches/buttons through 2 flip-flops; JB7-10 (CS1#, RST#, WP#, HOLD#) and JD7-10 (probe copies of UART TXD/RXD, I2C SCL/SDA)
`zed2_top_ps.v`	Way 2 top: the generated block-design wrapper (PS + `ssc2_axi_top`) and the pads
`zed2_top_standalone.v`	Way 1 top: reset synchroniser (BTNC), `ssc2_boot`, `ssc2_core`, `ssc2_axi_bram`, pads
`zed2_pins.xdc`	pins of both designs: JA-JD top rows = crossbar ports 0-3, OLED, LEDs, switches, pull-ups, false paths
`zed2_buttons_ps.xdc`, `zed2_standalone.xdc`	buttons (Way 2) / 100 MHz clock and BTNC reset (Way 1)

4. Software (`sw/`)

file	what it does
`ssc2_regs.h`	every register offset and bit field (C names used everywhere)
`ssc2_driver.c/.h`	register access, interrupts (GIC, ISR collecting INT_STATUS and INT2_STATUS), waiting without bus traffic (variables + WFI, CPU global timer), the v1 engine functions (UART, SPI + flash, I2C + TMP2), and the v2 set-up functions: crossbar, NIs, hub tasks, DMA ring with cache maintenance, serial-engine load/config/FIFOs
`main.c`	the console: `info`, `id`, `temp` (v1 way + CPU load), `flowa`, `hub` (v2 way + CPU load), `switch`, `se ...`, `regs`
`se_demos.c/.h`	one demo per program: `se uart`, `se spi`, `se i2c`, `se man` (Manchester + CRC), `se oled`, and the slide-14 `switch` demo
`se_programs.h`	generated by `tools/build_programs.sh` from `programs/*.se`
`oled_font.h`	generated by `tools/make_font.py`: 5x7 font

5. Tools (`tools/`)

file	what it does
`se_asm.py`	the serial-engine assembler: `.se` -> `.hex`, `.lst`, C header (labels, directives, side-set, delays, range checks, clear error messages)
`build_programs.sh`	assembles every program and regenerates `sw/se_programs.h`
`pc_frame.py`	PC side of flow A: sends a frame to the UART NI and prints the reply (`01 9F 03` -> `20 BA 19`)
`make_font.py`	builds `sw/oled_font.h`
`area_yosys.sh`	area estimate per block without Vivado

6. Trace 1: one temperature reading, start to finish (slide 19)

Set-up (once): the CPU (or `ssc2_boot`) writes hub task 0 = "every 100 ms, XFER_REQ to node 5, arg 0x4B, write `00`, read 2 bytes, make a record", enables the I2C NI and the DMA writer.

1. **Timer.** `ssc2_timebase` pulses `ms_tick`; task 0's countdown in `hub` reaches 0 and the task becomes due.
2. **Request.** `hub` records `t_start = time_us` and gives `noc_pkt_tx` a header (type 1, dest 5, src 1, len 3, tag {roll, 0}, arg 0x4B) and the payload `01 02 00` (wlen, rlen, pointer). That is 1 HEAD + 1 TAIL flit.
3. **Network.** Router (1,0) sends it EAST (dest column 2 > 1), router (2,0) SOUTH (row 1 > 0), router (2,1) LOCAL: 3 routers x 2 clocks.
4. **I2C NI.** `noc_pkt_rx` gives `ni_i2c` the header; it raises `active` (the core forces the I2C engine to 7-bit address 0x4B), pushes `00`, issues `cmd(len 1, write, no STOP)`, waits for `ev_done`, issues `cmd(len 2, read, STOP)`, pops `0C 80`.
5. **I2C engine.** `ssc_i2c` drives SCL/SDA as open drain; the crossbar routes them to JC3/JC4 (port 2 = I2C after reset). About 120 us at 400 kHz, 470 us at 100 kHz.
6. **Reply.** `ni_i2c` sends XFER_RESP `[00][0C][80]` to node 1 with the same tag: WEST to (1,1), NORTH to (1,0), LOCAL.
7. **Record.** `hub` matches the tag, keeps `0C 80` as the last value, and sends a RECORD to node 2: `{t_start, 3 (I2C), 2, seq, 0C 80 00 ...}`, 16 bytes = 1 HEAD + 4 flits, one hop EAST.
8. **DDR.** `dma_writer` collects the 16 bytes and writes one burst (AWLEN 3) to `BASE + 16 x WR_IDX` through HP0; WR_IDX and PENDING go up.
9. **Batch.** At PENDING = 32, `ev_batch` sets INT2 bit 0; `irq` rises; the ARM wakes from WFI, invalidates the 32 records in its cache, reads them, writes DMA_RD_IDX, and sleeps again.

Bus traffic during steps 1-8: none. One interrupt every 32 readings.

7. Trace 2: the PC reads the flash ID (slide 17, flow A)

1. The PC sends `01 9F 03` to the PmodUSBUART on JA. `ssc_uart` receives the bytes into its RX FIFO (JA3 -> crossbar port 0 pin 2 -> RXD).
2. `ni_uart` (FRAME mode, dest 4) parses `[wlen=1][9F][rlen=3]` and sends XFER_REQ `[01][03][9F]` from node 3 to node 4: one hop EAST.
3. `ni_spi` raises `active` (the core forces 8-bit words, manual CS, master), asserts CS0#, pushes `9F 00 00 00` into the SPI TX FIFO, keeps the last 3 received bytes `20 BA 19`, waits for `!busy`, releases CS0#.
4. `ni_spi` replies XFER_RESP `[00][20][BA][19]` to node 3: one hop WEST.
5. `ni_uart` pushes `20 BA 19` into the UART TX FIFO (the status byte is skipped unless RESP_STATUS is set); the PC prints `20 BA 19`.

At the same time flow B can use router (1,1) on other links (slide 17); `tb_ssc2_system` test [4] runs both at once.

8. Where to look next

- The detailed design of a block: `docs/blocks/<block>.md`.
- Every register: `docs/REGISTER_MAP.md`.
- How it is tested and what correct output looks like: `docs/VERIFICATION.md`.
- The slide-24 measurements: `docs/MEASUREMENTS.md`.
- Building and running it: `GETTING_STARTED.md`.